National University of Computer & Emerging Sciences

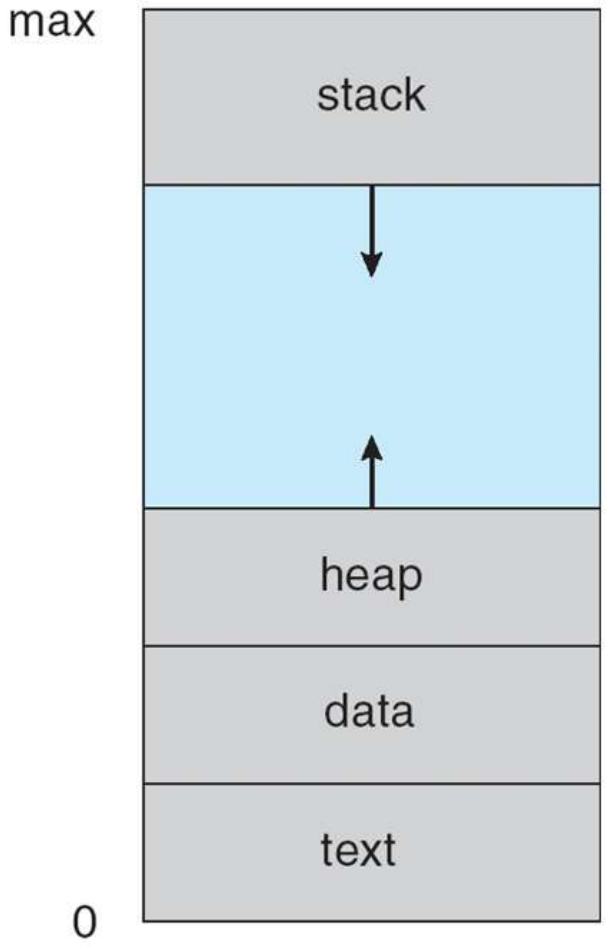
Lecture 7

Process Control Block

Process

- A computer program in execution on a machine is a process
- More formally:
  - A **Sequential stream of Execution** in its own **address space**

Process in Memory



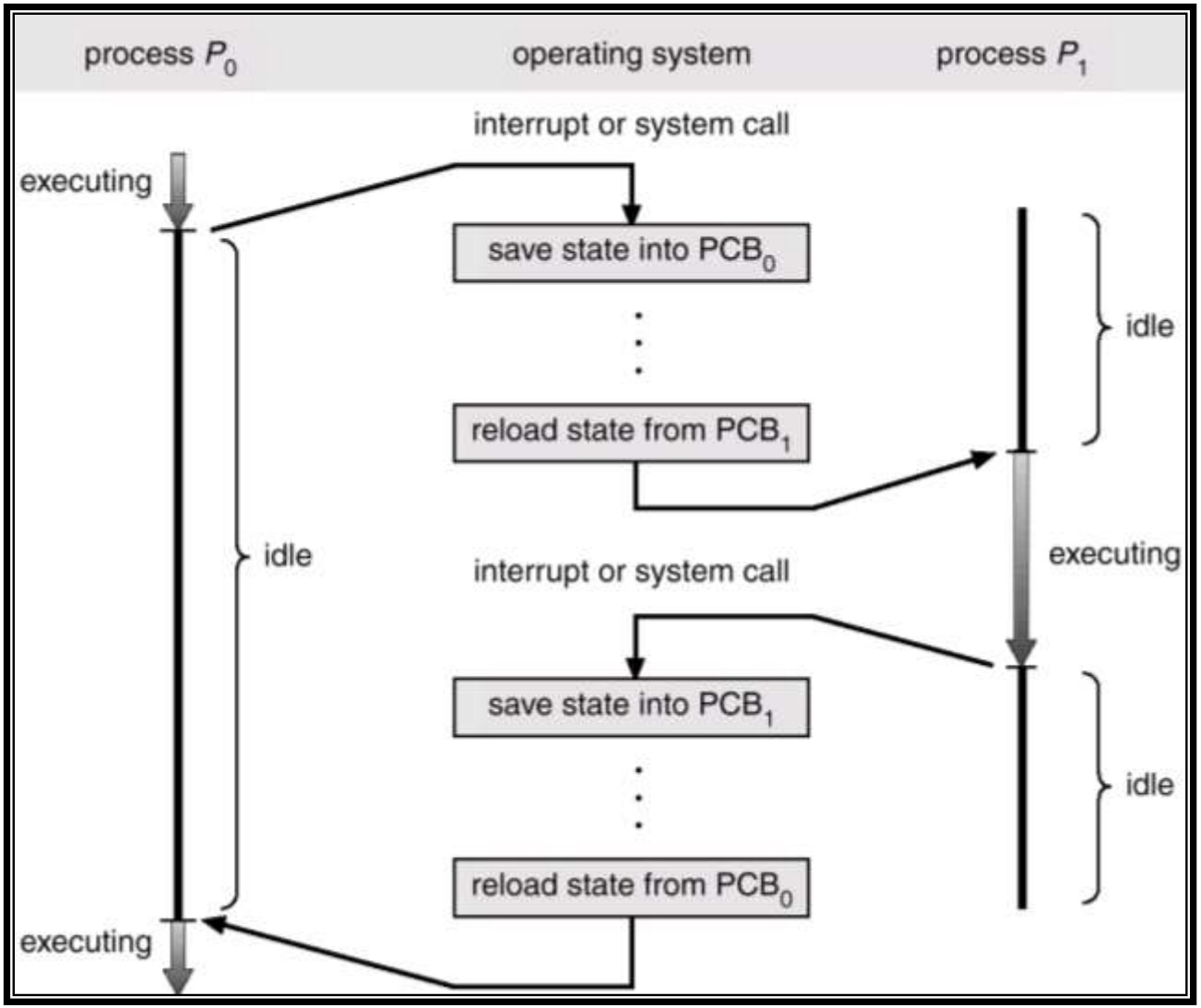
PCB

- In general a PCB includes:
  - The identifier of the process (a process identifier, or PID)
  - Register values for the process including, notably, the program counter and stack pointer values for the process.
  - The address space for the process
  - Priority (in which higher priority process gets first preference. e.g., nice value on Unix operating systems)
  - Process accounting information, such as when the process was last run, how much CPU time it has accumulated, etc.
  - I/O Information (i.e. I/O devices allocated to this process, list of opened files, etc.)

Process Control Block (PCB)

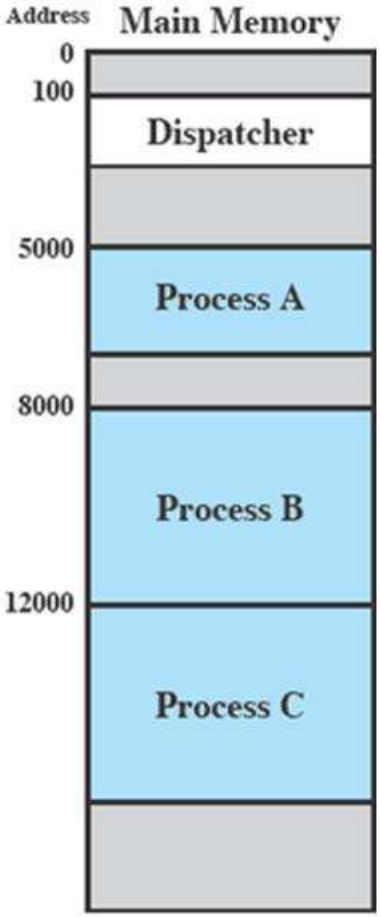


CPU Switch From Process to Process



CPU Switch From Process to Process

- Switching a process requires
  - Saving the state of old process
  - Loading the saved state of the new process
- This is called **Context Switch**
- Part of OS responsible for switching the processor among the processes is called **Dispatcher**

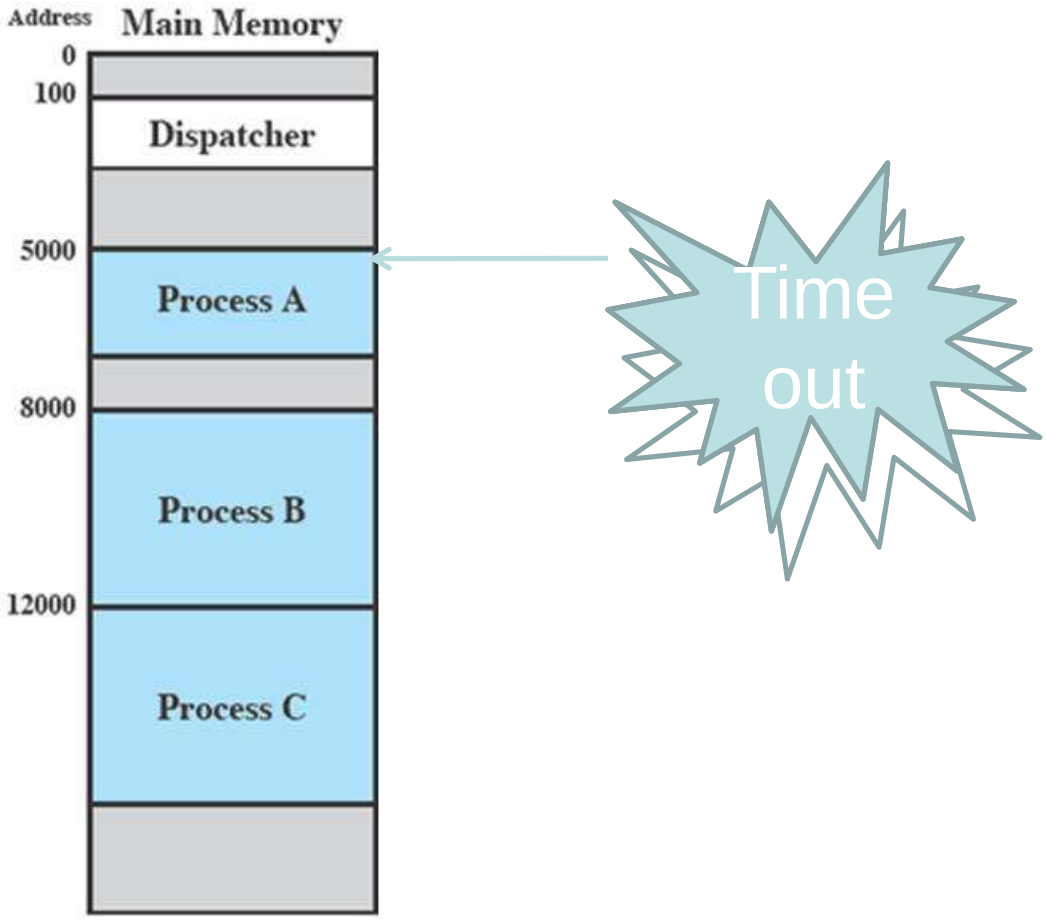


(a) Trace of Process A	(b) Trace of Process B	(c) Trace of Process C
5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C

Figure 3.3 Traces of Processes of Figure 3.2

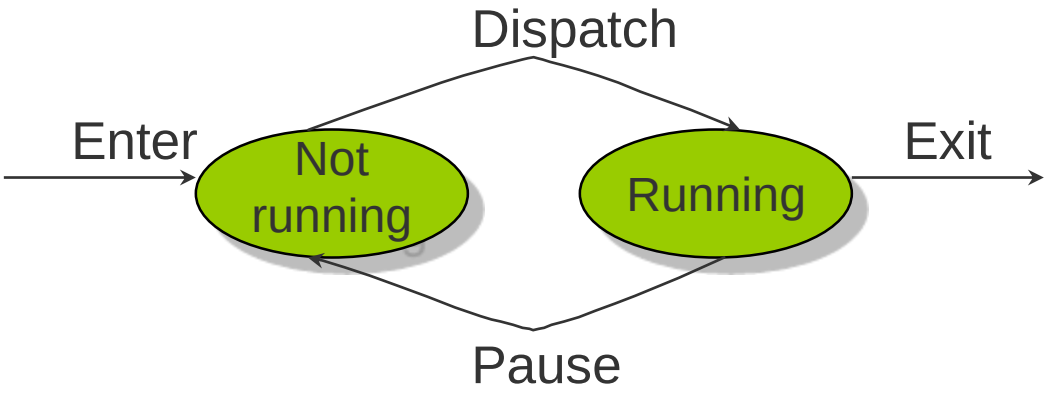
Trace from Processors point of view



Process States

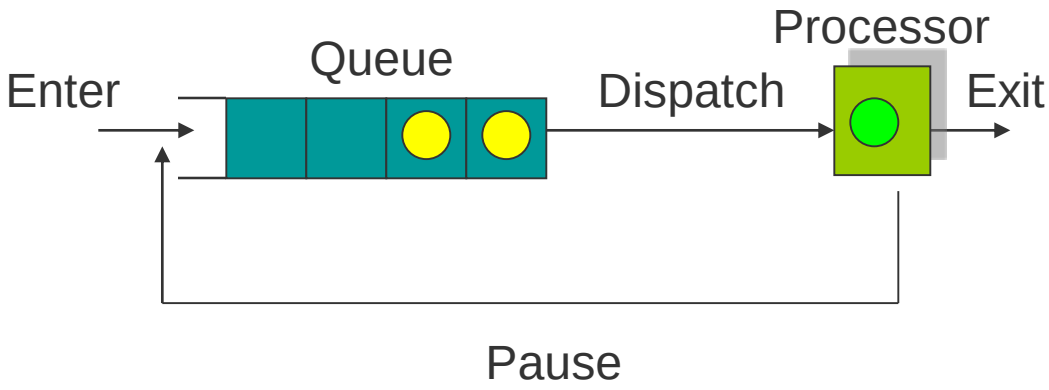
- At any given time a process is either running or not running
- Number of states
  - Running
  - Not Running
- When the OS creates a process, the process is entered into which state?
  - Not Running

Two-state process model



- Number of processes **Running** at a particular time?
- Number of processes **Not Running** at a particular time?
- Data Structure?
- Processes that are **Not Running** at a particular time should be kept in some sort of a **queue**

Two-state process model



Dispatcher is now redefined:

- Moves processes to the waiting queue
- Remove completed/aborted processes
- Select the next process to run

How process state changes1

Ready

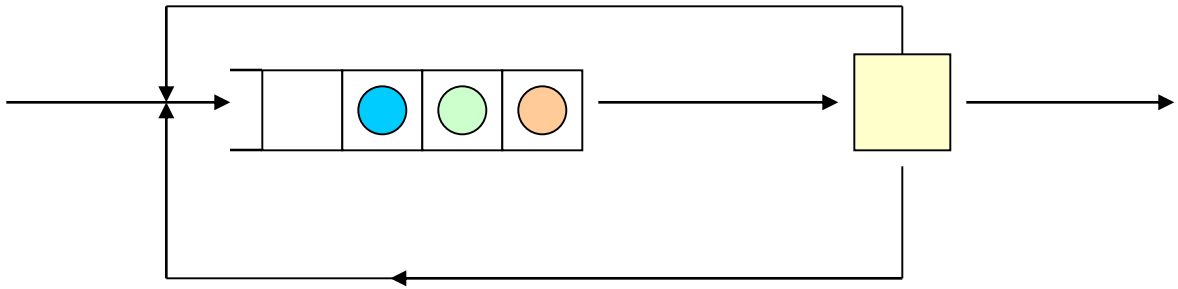
➔ a := 1
b := a + 1
c := b + 1
read a file
a := b - c
c := c * b
b := 0

Ready

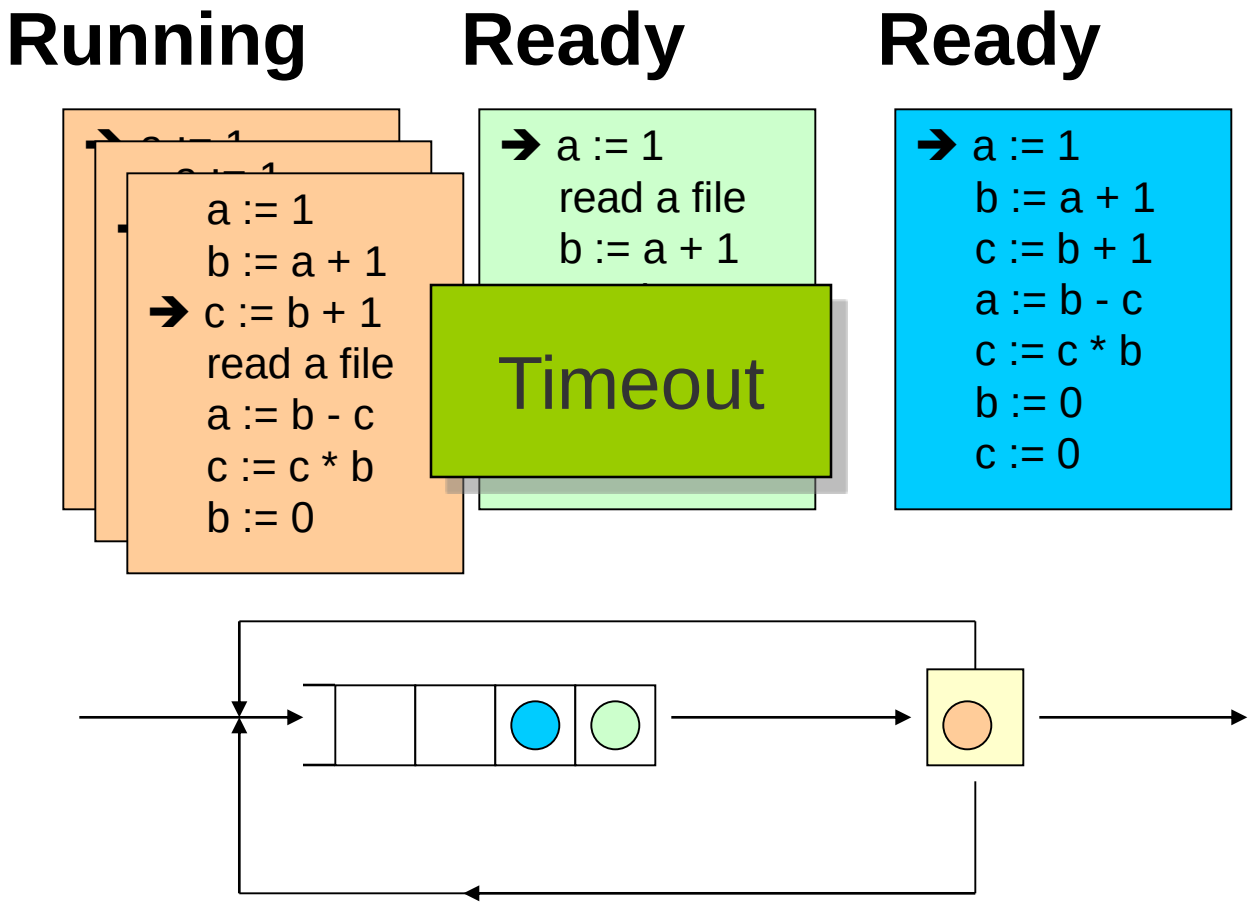
➔ a := 1
read a file
b := a + 1
c := b + 1
a := b - c
c := c * b
b := 0

Ready

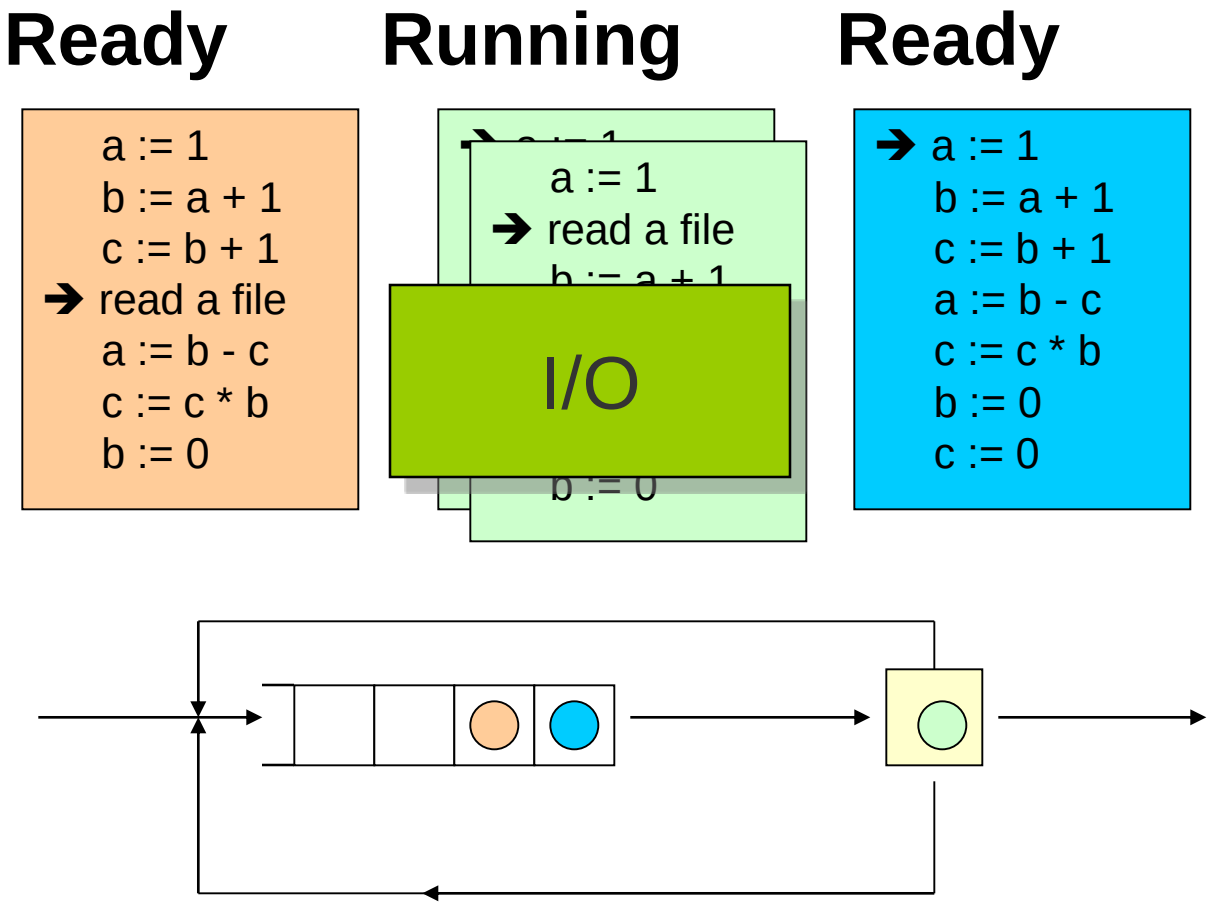
➔ a := 1
b := a + 1
c := b + 1
a := b - c
c := c * b
b := 0
c := 0



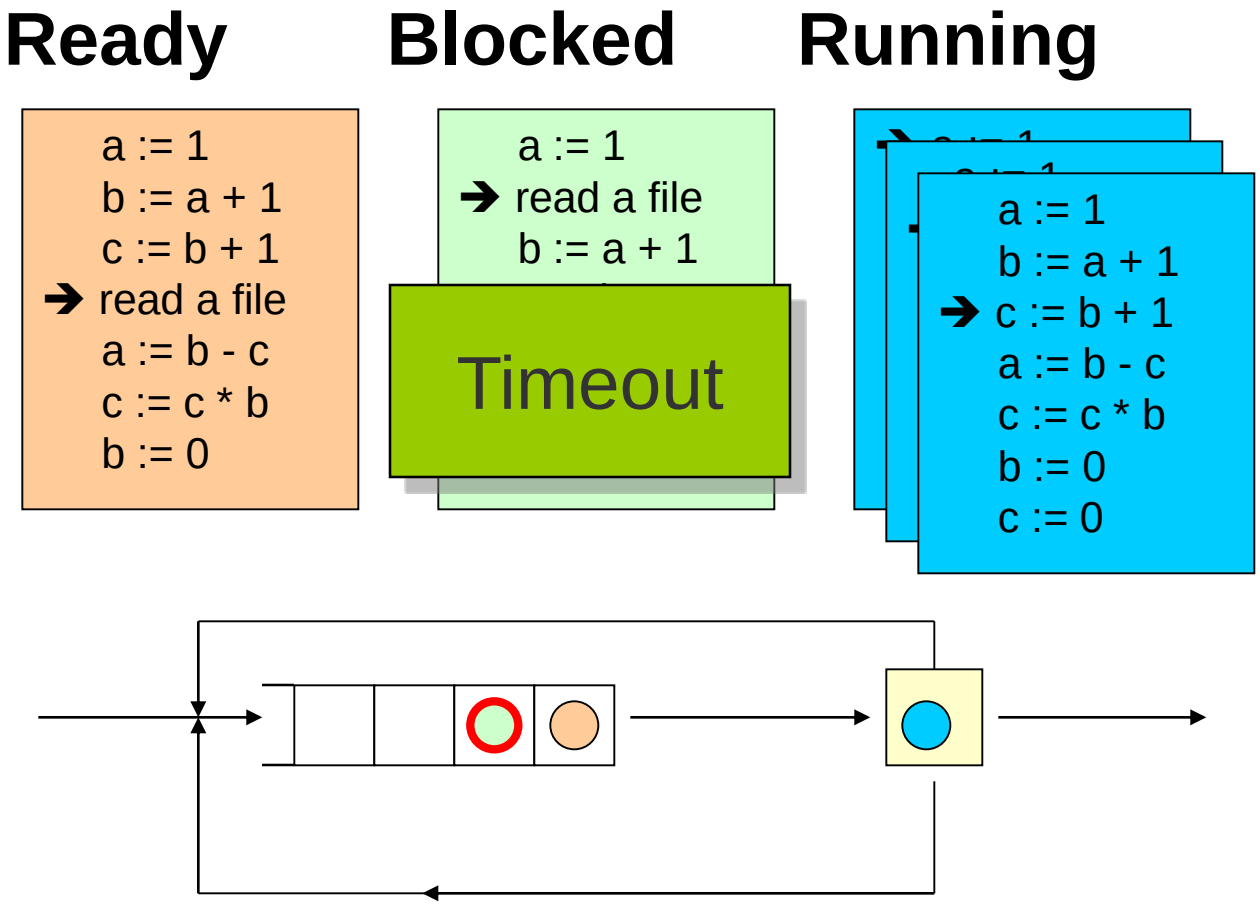
How process state changes2



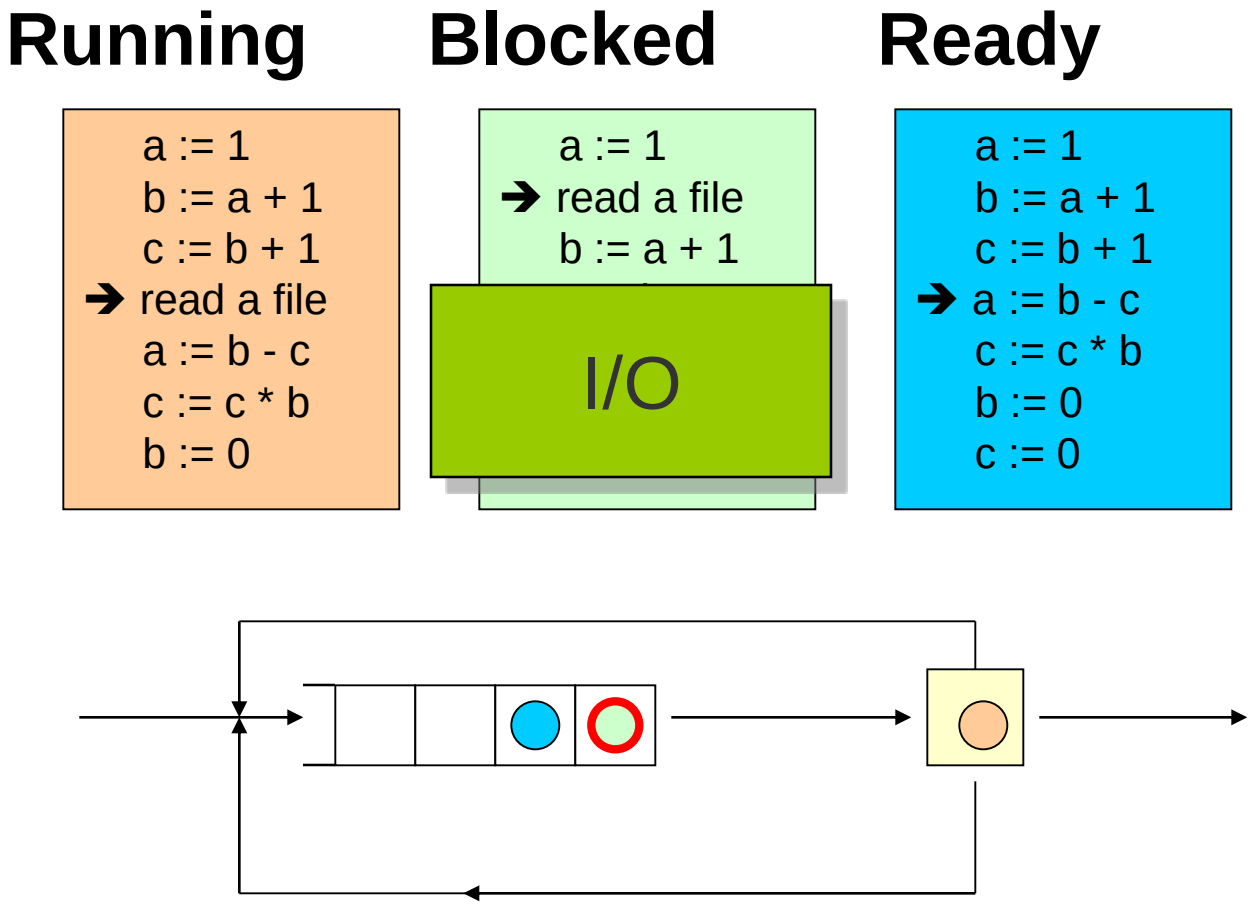
How process state changes3



How process state changes4



How process state changes5



How process state changes6

Blocked

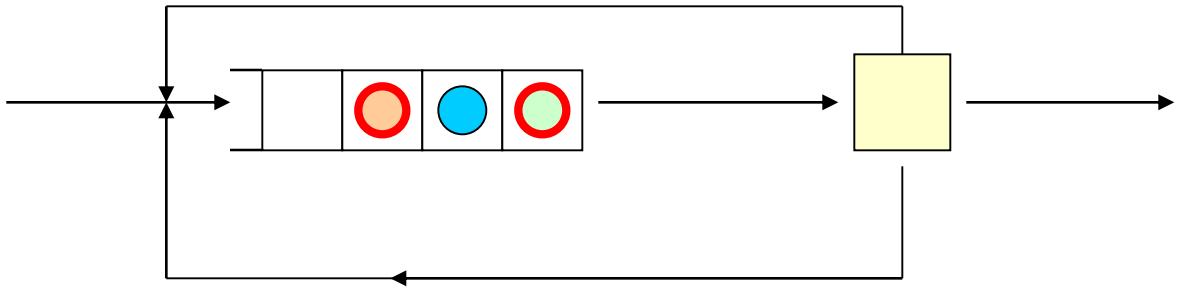
```
    a := 1
    b := a + 1
    c := b + 1
➔ read a file
    a := b - c
    c := c * b
    b := 0
```

Blocked

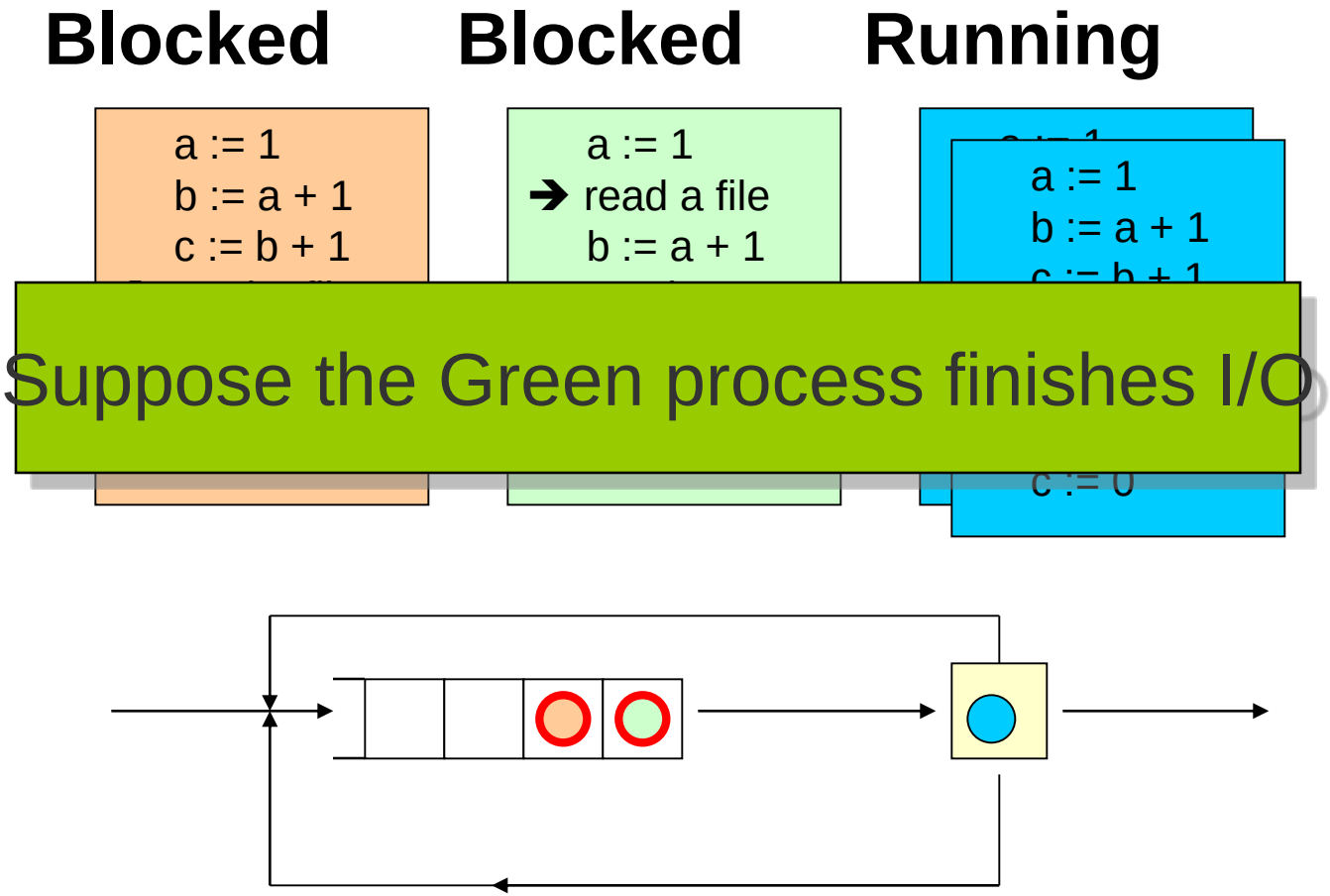
```
    a := 1
➔ read a file
    b := a + 1
    c := b + 1
    a := b - c
    c := c * b
    b := 0
```

The Next Process to Run cannot be simply selected from the front

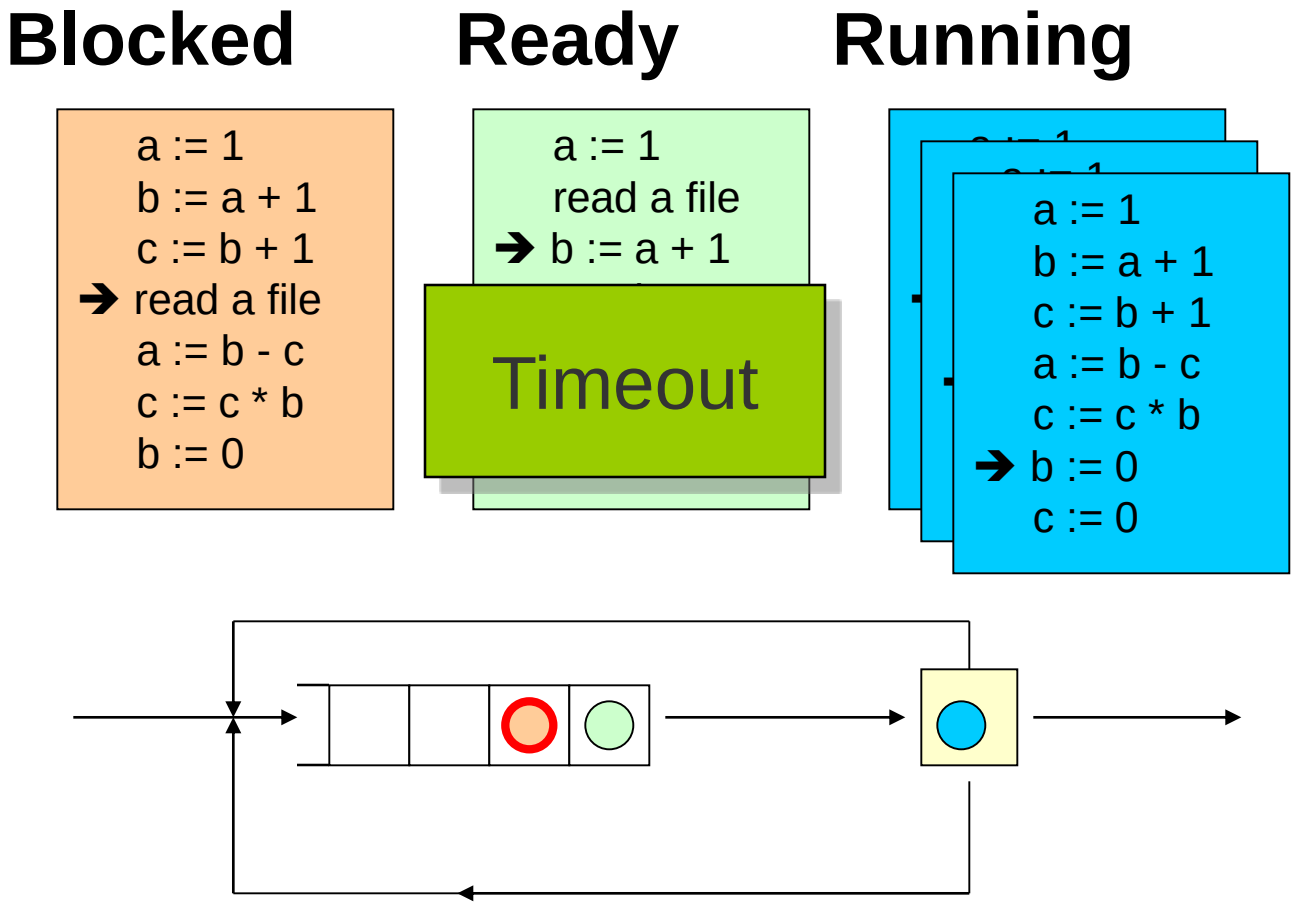
```
➔ a := b - c
    c := c * b
    b := 0
    c := 0
```



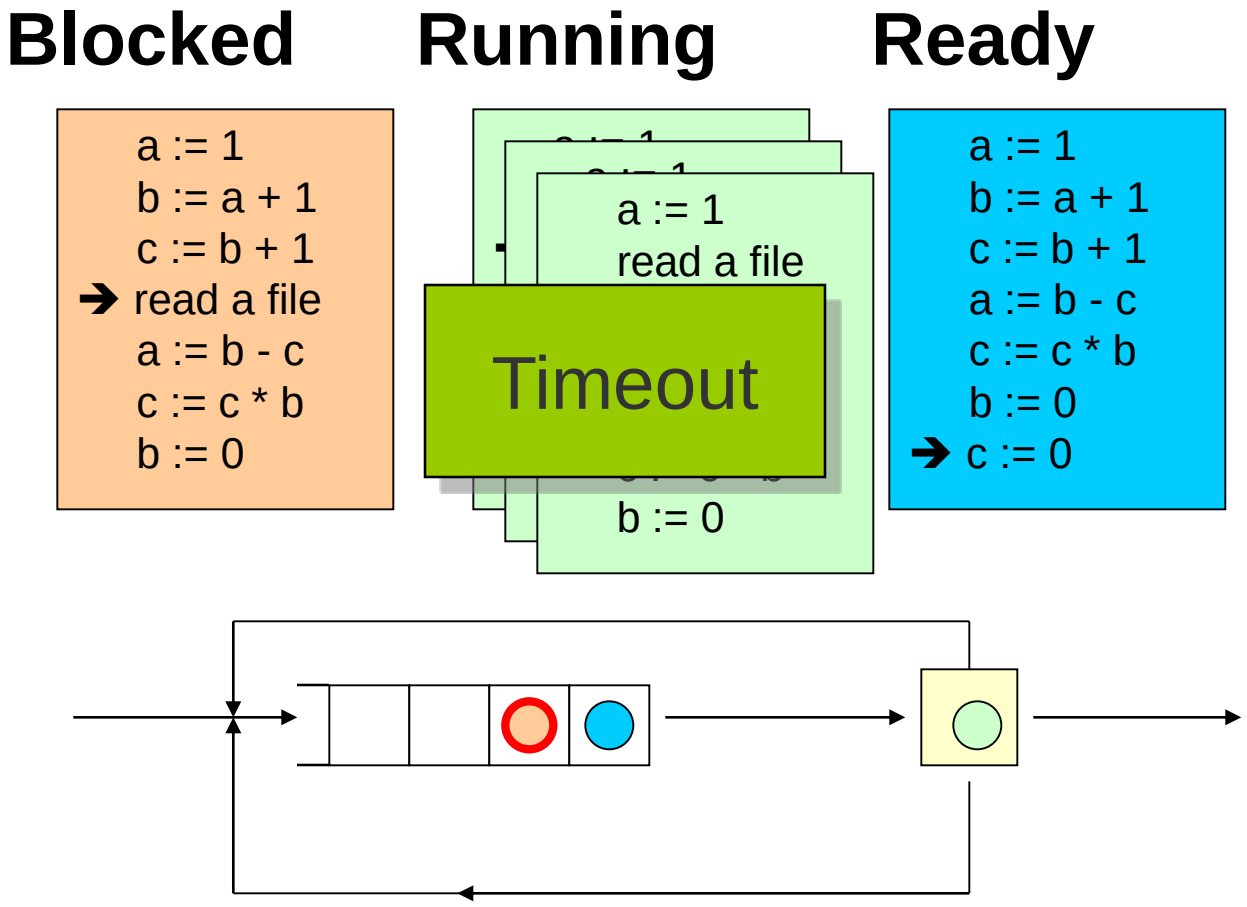
How process state changes7



How process state changes8



How process state changes9



Problem in Two-state Process model

- A process may be waiting for I/O request
- A single queue for both the ready to run and waiting processes
- The dispatcher cannot simply select the process at the front, it can be a busy process
- In the worst case, it has to scan the whole queue to find the next process to run
- Solution?
- Split the Not Running state to:
  - Waiting, Ready

Five-state Process Model

- Running: currently being run
- Ready: ready to run
- Blocked: waiting for an event (I/O)
- New: just created, not yet admitted to set of run-able processes
- Exit: completed/error exit

Five-state Process Model

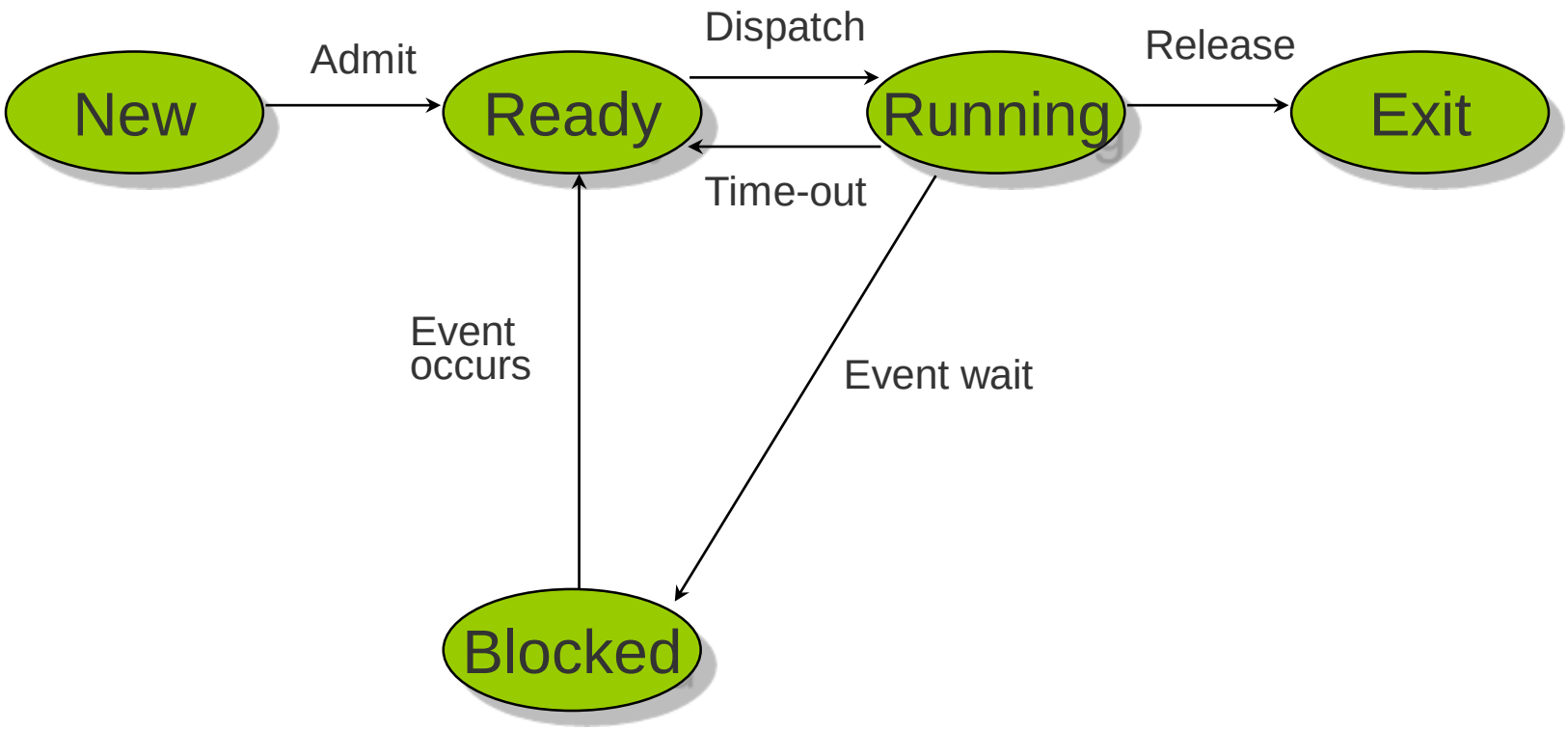
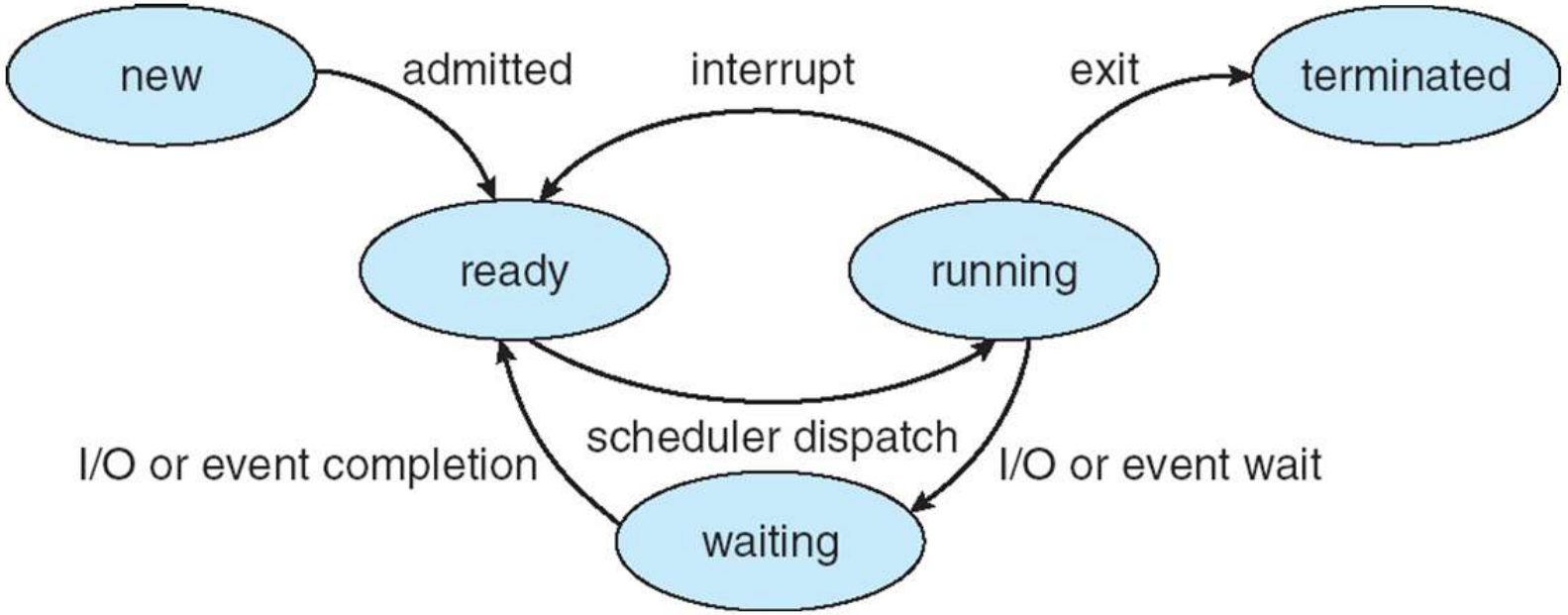
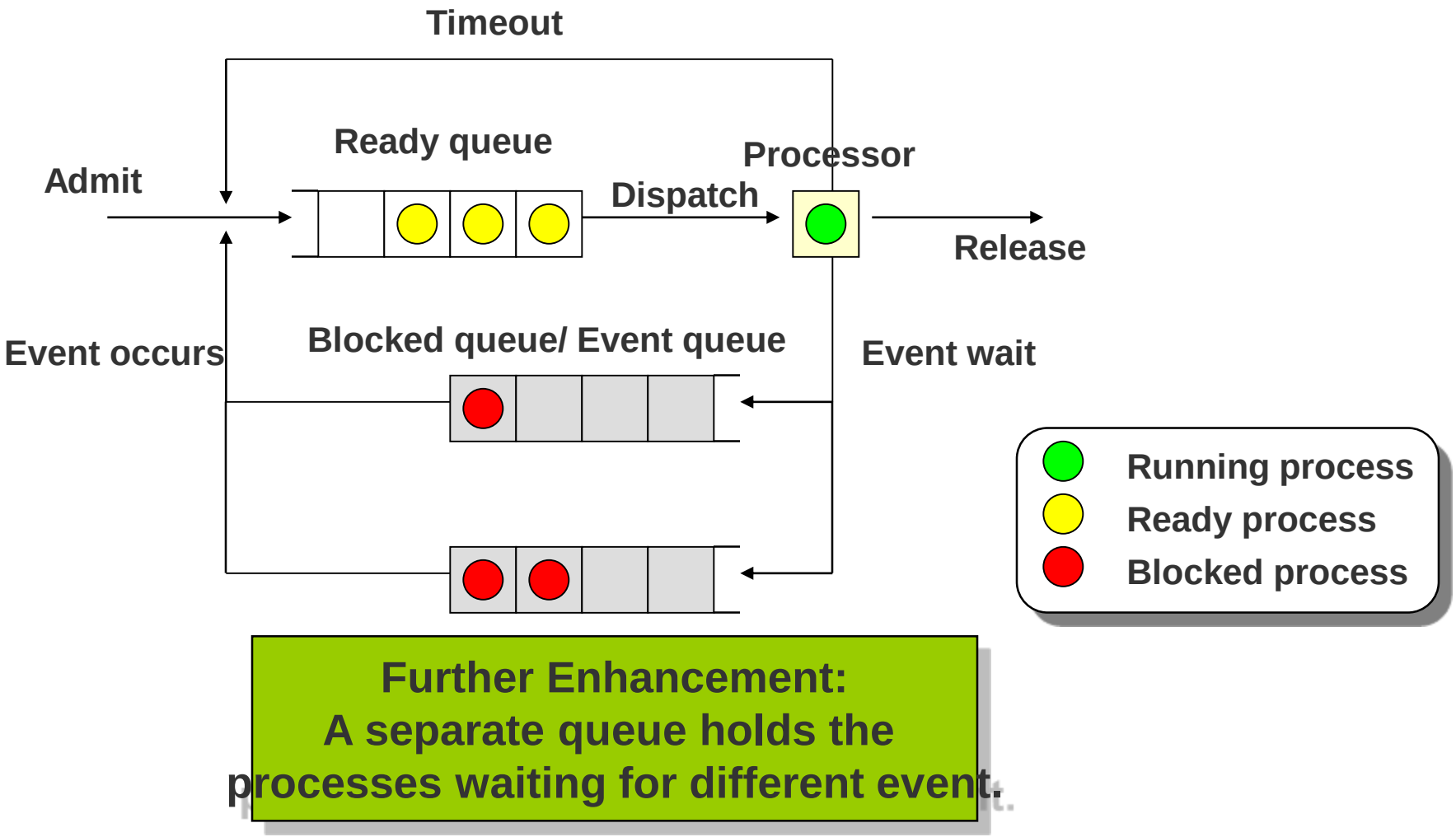


Diagram of Process State



Blocked Queues

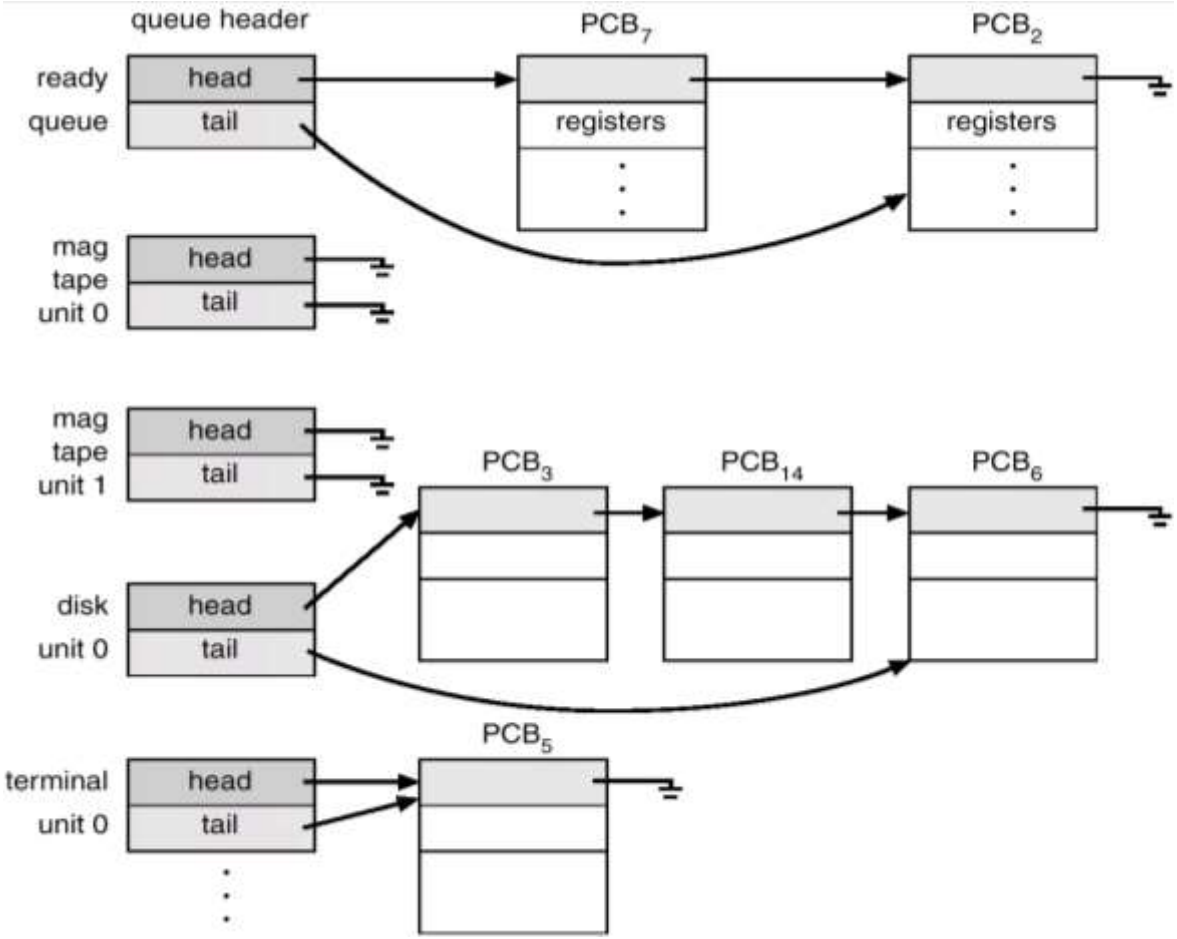


Further Enhancement:
A separate queue holds the processes waiting for different event.

Scheduling Queues

- The queues are generally stored as linked lists
- A queue header points to the first and the final PCB's in the list
- We extend each PCB to include a pointer field that points to the next PCB in the ready queue

Scheduling Queues



Schedulers

- Short term Scheduler or CPU Scheduling
  - Which program is to be run next
- Long term Scheduler or Job Scheduler
  - Which ready jobs should be brought to memory
  - May need to invoke only when a process leaves the system
  - Must make a careful selection

Process Types

- Most processes can be described as either I/O bound or CPU bound
- I/O bound:
  - Spends more of its time doing I/O than doing computations
- CPU bound:
  - Spends more of its time doing computations than doing I/O
- If all processes are I/O bound,
  - The ready queue will almost always be empty
- If all processes are CPU bound,
  - The I/O waiting queue will almost always be empty, devices will go unused
  - System will again be unbalanced

Process Types

- Sometimes OS may swap a blocked process to disk to free up more memory
- Or to improve process mix
- This is called **Swapping**